

PROYECTO BIG DATA

# Radar electoral de comentarios en vivo

Informe de arquitectura, procesamiento NLP  
y operación distribuida

|            |                                   |
|------------|-----------------------------------|
| Alumno     | Sixto Caceres Terrones            |
| Plataforma | AWS EMR, S3, Kafka, Flink y Spark |
| Interfaz   | Dashboard web local               |
| Fecha      | 30 de junio de 2026               |
| Versión    | Arquitectura distribuida vigente  |

# Índice general

|                                                        |            |
|--------------------------------------------------------|------------|
| <b>Resumen ejecutivo</b>                               | <b>III</b> |
| <b>1. Arquitectura del clúster</b>                     | <b>1</b>   |
| 1.1. Topología física . . . . .                        | 1          |
| 1.2. Roles y disponibilidad . . . . .                  | 1          |
| <b>2. Arquitectura detallada del pipeline</b>          | <b>2</b>   |
| 2.1. Diagrama de componentes . . . . .                 | 2          |
| 2.2. Flujo de una sesión . . . . .                     | 2          |
| 2.3. Topics y contratos . . . . .                      | 3          |
| <b>3. Instalación, configuración y arranque</b>        | <b>4</b>   |
| 3.1. Prerrequisitos . . . . .                          | 4          |
| 3.2. Configuración . . . . .                           | 4          |
| 3.3. Arranque desde cero . . . . .                     | 4          |
| 3.4. Validación . . . . .                              | 4          |
| 3.5. Scripts después de reiniciar una sesión . . . . . | 5          |
| 3.6. Parada segura . . . . .                           | 5          |
| <b>4. Datasets y fuentes</b>                           | <b>6</b>   |
| 4.1. YouTube Live Chat electoral peruano . . . . .     | 6          |
| 4.2. OffendES . . . . .                                | 6          |
| 4.3. Consideraciones éticas . . . . .                  | 6          |
| <b>5. Técnicas NLP</b>                                 | <b>8</b>   |
| 5.1. Normalización streaming . . . . .                 | 8          |
| 5.2. Reglas lingüísticas locales . . . . .             | 8          |
| 5.3. Spark ML con OffendES . . . . .                   | 8          |
| 5.4. Scoring híbrido . . . . .                         | 8          |
| <b>6. Documentación de los 10 jobs</b>                 | <b>9</b>   |
| 6.1. Idempotencia y control . . . . .                  | 9          |
| 6.2. Cinco jobs Flink . . . . .                        | 9          |
| 6.3. Cinco jobs Spark . . . . .                        | 9          |
| <b>7. Reporte de métricas</b>                          | <b>10</b>  |
| 7.1. Validación distribuida de 2,000 eventos . . . . . | 10         |
| 7.2. Métricas ML . . . . .                             | 10         |
| 7.3. Métricas del corpus completo . . . . .            | 10         |
| 7.4. Cadencia del dashboard . . . . .                  | 11         |
| <b>8. Interfaz gráfica funcional</b>                   | <b>12</b>  |

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>9. Capturas del dashboard en operación</b>                   | <b>13</b> |
| 9.1. Vista general y chat en vivo . . . . .                     | 13        |
| 9.2. Eventos Flink y comentarios filtrados por reglas . . . . . | 14        |
| 9.3. Procesamiento Spark batch y filtrado ML . . . . .          | 14        |
| <b>10. Conclusiones</b>                                         | <b>16</b> |
| <b>A. Trazabilidad documental</b>                               | <b>17</b> |
| <b>B. Rutas principales</b>                                     | <b>18</b> |

# Resumen ejecutivo

El sistema analiza comentarios de YouTube Live Chat asociados al discurso electoral peruano. La fuente durable reside en Amazon S3; un productor Python reproduce los registros como flujo y los publica en un clúster Kafka KRaft de tres nodos. Desde el topic `raw_youtube_chat`, cinco jobs Flink realizan análisis de baja latencia y Spark procesa rangos disjuntos de 1,000 registros. Los resultados streaming regresan a Kafka y los productos batch se almacenan como Parquet en S3 Curated.

La solución combina dos familias de técnicas NLP: reglas lingüísticas específicas del contexto peruano y modelos Spark ML entrenados con OffendES. Las reglas aportan señales explicables de terruqueo, fraude electoral, instituciones, actores, insultos y discriminación; el modelo aporta clasificación general de ofensividad en español. El nivel de riesgo híbrido evita interpretar la predicción ML cruda como una verdad política.

La validación distribuida registró quorum Kafka de tres voters, cero particiones offline, cinco consumidores Flink, batches Spark sin solapamiento y salidas S3 separadas por rango. El dashboard ofrece control de arranque y parada, feeds en vivo, contadores, batches y diagnóstico. Mientras existe una sesión AWS activa, el estado principal se muestra como **conectado** con foco verde; las advertencias técnicas permanecen visibles en el panel detallado.

# Capítulo 1

## Arquitectura del clúster

### 1.1. Topología física

La plataforma separa el bus de eventos del cómputo. Esta decisión evita que una aplicación Spark intensiva compita directamente con Kafka por memoria y CPU.

| Capa      | Nodo o clúster  | Rol                    | Servicios                                                                |
|-----------|-----------------|------------------------|--------------------------------------------------------------------------|
| Datos     | Amazon S3       | Fuente durable         | CSV Raw, modelos OffendES, Parquet Curated, reportes y agregados.        |
| Eventos   | Primary, nodo 1 | Entrada y coordinación | Broker/controller 1, producer Python, monitor, SSH y descubrimiento EMR. |
| Eventos   | Core, nodo 2    | Réplica Kafka          | Broker/controller 2, réplica e ISR.                                      |
| Eventos   | Core, nodo 3    | Réplica Kafka          | Broker/controller 3, réplica e ISR.                                      |
| Streaming | Primer worker   | Baja latencia          | Flink 1.17.1, cinco jobs, YARN y acceso privado a Kafka.                 |
| Batch     | Workers compute | Cómputo por rangos     | Spark 3.4.1 sobre YARN, launchers, locks y estados JSON atómicos.        |
| Control   | Máquina local   | Plano de control       | Node.js, API HTTP, dashboard, SSH/SCP y llave PEM local.                 |

### 1.2. Roles y disponibilidad

- Los tres nodos Kafka combinan los roles `broker`, `controller`. El quorum utiliza puerto 9093 y el tráfico de clientes usa 9092.
- Los topics operativos tienen tres particiones, factor de replicación 3 y `min.insync.replicas=2`.
- El primary descubre dinámicamente los DNS privados de los core nodes; la llave privada no se copia a AWS.
- Flink usa grupos estables. Si un job cae, su offset confirmado permite medir el lag y reanudar.
- Spark mantiene rangos inmutables e idempotentes. La política segura es `SPARK_MAX_CONCURRENCY=1`.

## Capítulo 2

# Arquitectura detallada del pipeline

### 2.1. Diagrama de componentes

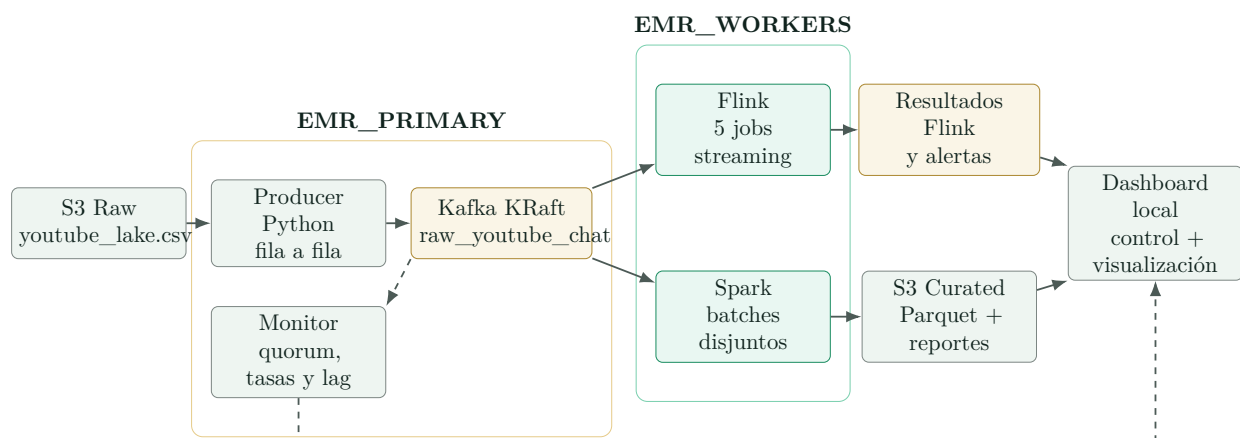


Figura 2.1: Pipeline distribuido vigente. Flink y Spark son consumidores independientes del mismo topic RAW.

### 2.2. Flujo de una sesión

1. El operador inicia el dashboard y pulsa **Conectar AWS**.
2. El bootstrap descubre tres nodos Kafka, prepara KRaft y crea los topics.
3. El productor lee el CSV de S3, añade `event_id` y `row_number`, y publica en Kafka.
4. Flink procesa continuamente y confirma offsets por grupo.
5. Al existir un múltiplo completo de 1,000 eventos, Spark agenda el rango siguiente.
6. Spark valida el tamaño exacto, aplica reglas, ML e híbrido, y escribe rutas exclusivas en S3.
7. El dashboard consulta deltas y salud sin consumir ni modificar los offsets de los jobs.

## 2.3. Topics y contratos

| Topic               | Productor       | Contenido                                                                          |
|---------------------|-----------------|------------------------------------------------------------------------------------|
| raw_youtube_chat    | Producer Python | Comentario original, metadatos de origen, número de fila y timestamp de ingestión. |
| nlp_stream_results  | Flink 1–4       | Normalización, métricas por ventana, señales políticas y polarización por actor.   |
| alerts_polarization | Flink 5         | Alertas explicables con tipo, severidad, actor, reglas y puntaje local.            |
| nlp_batch_results   | Reservado       | Integración batch opcional; la salida vigente de Spark se conserva en S3.          |

## Capítulo 3

# Instalación, configuración y arranque

### 3.1. Prerrequisitos

- EMR\_PRIMARY con tres instancias RUNNING.
- Uno o más EMR\_WORKERS con Hadoop, Spark 3.4.1 y Flink 1.17.1.
- Conectividad privada entre clústeres y permisos S3.
- Node.js 18 o superior, PowerShell o Git Bash, SSH/SCP y AWS CLI.
- `final.pem` y un archivo `.env` local, ambos fuera de Git.

### 3.2. Configuración

```
EMR_PRIMARY=ec2-primary.compute-1.amazonaws.com
EMR_WORKERS=ec2-compute-1.compute-1.amazonaws.com
DATA_SIZE=30000
SPARK_BATCH_SIZE=1000
SPARK_MAX_CONCURRENCY=1
```

### 3.3. Arranque desde cero

```
cd emr_kafka_setup/dashboard
./start_dashboard.sh
# Windows: .\start_dashboard.ps1
```

Se abre <http://127.0.0.1:8787> y se pulsa **Conectar AWS**. Para un bootstrap directo y controlado:

```
./scripts/bootstrap_emr_streaming.sh \
--limit 2000 \
--producer-delay-ms 10 \
--reset-topics
```

### 3.4. Validación

```
./scripts/aws_status_from_aws.sh \
--data-size 2000 \
--spark-batch-size 1000 \
--spark-max-concurrency 1
```



```
./scripts/spark_status_from_aws.sh --max-concurrency 1
./scripts/pipeline_health_from_aws.sh
```

### 3.5. Scripts después de reiniciar una sesión

Si solo se cerró la terminal local, se vuelve a ejecutar el dashboard. Si también deben restaurarse producer, monitor y jobs, conservando topics y offsets:

```
./scripts/restart_services_after_session.sh
# Windows:
.\scripts\restart_services_after_session.ps1
```

Estos wrappers invocan `bootstrap_emr_streaming.sh --no-reset-topics`. Para empezar una sesión limpia se usa **Detener plataforma** y luego **Conectar AWS**.

### 3.6. Parada segura

```
./scripts/stop_emr_streaming.sh
```

La parada cancela aplicaciones YARN, drivers Spark, jobs Flink, producer, monitor y brokers. No elimina el CSV, los modelos ni los productos S3.

## Capítulo 4

# Datasets y fuentes

### 4.1. YouTube Live Chat electoral peruano

El dataset principal contiene 160,464 comentarios deduplicados. Fue recolectado de chats públicos de YouTube asociados a transmisiones y discusión electoral peruana mediante `yt-dlp`. El archivo durable es:

```
s3://figuretibucket/data/raw/youtube/youtube_lake.csv
```

Las columnas relevantes se organizan así:

| Columna                                  | Uso en el pipeline                                 |
|------------------------------------------|----------------------------------------------------|
| <code>source_file</code>                 | Archivo de chat del que proviene el registro.      |
| <code>video_id</code>                    | Identificador de la transmisión de YouTube.        |
| <code>timestamp_text</code>              | Marca temporal legible dentro del chat.            |
| <code>timestamp_usec</code>              | Tiempo original en microsegundos.                  |
| <code>video_offset_msec</code>           | Posición relativa usada para agregados históricos. |
| <code>author</code>                      | Autor público del mensaje.                         |
| <code>message_raw / message_clean</code> | Texto original y versión preparada para análisis.  |

La fuente no incluye etiquetas humanas de ofensividad; por esa razón no se usa como corpus supervisado de entrenamiento.

### 4.2. OffendES

OffendES es el corpus externo etiquetado para investigación de lenguaje ofensivo en español. La copia de trabajo se conserva en S3:

```
s3://figuretibucket/dataset/offendES/train.parquet
s3://figuretibucket/dataset/offendES/validation.parquet
s3://figuretibucket/dataset/offendES/test.parquet
```

Fuentes públicas: <https://huggingface.co/datasets/fmplaza/offendes> y el artículo <https://aclanthology.org/2021.ranlp-1.123/>. El mapeo binario usa etiquetas 0/1 como ofensivo y 2/3 como no ofensivo. El modelo multiclase conserva las cuatro categorías analíticas.

### 4.3. Consideraciones éticas

- Las reglas y predicciones son señales analíticas, no juicios definitivos sobre personas.
- Términos étnicos, regionales o políticos requieren contexto humano.

- Los identificadores de usuarios no deben usarse para persecución, perfilado sensible ni decisiones automatizadas.
- El cambio de dominio entre OffendES y chat político peruano obliga a reportar incertidumbre.

## Capítulo 5

# Técnicas NLP

### 5.1. Normalización streaming

Flink selecciona `message_clean` o el mensaje `raw`, convierte a minúsculas, reduce espacios y conserva trazabilidad con `topic`, partición y `offset`. La normalización es deliberadamente ligera para sostener baja latencia.

### 5.2. Reglas lingüísticas locales

Se aplican expresiones y diccionarios multilabel para terrorismo, fraude, instituciones electorales, actores, polarización, lenguaje discriminatorio, insultos y spam. Cada activación aporta etiquetas y un `local_risk_score`. La salida explica la razón del riesgo.

### 5.3. Spark ML con OffendES

El pipeline supervisado usa `RegexTokenizer`, eliminación de stopwords, `HashingTF`, IDF y regresión logística. Se entrenan un clasificador binario y otro multiclase. El entrenamiento es offline; la inferencia sobre cada batch carga modelos persistidos.

### 5.4. Scoring híbrido

La señal final combina confianza ML y reglas:

- **Bajo:** no hay evidencia fuerte.
- **Medio:** predicción incierta o reglas moderadas.
- **Alto:** confianza elevada y/o combinación local de riesgo.

Este enfoque compensa la sobredetección del modelo general y la cobertura limitada de reglas puramente locales.

## Capítulo 6

# Documentación de los 10 jobs

### 6.1. Idempotencia y control

Cada batch usa un identificador como `batch_0001000`. El launcher conserva un lock y un estado JSON remoto. Si el estado ya existe no vuelve a escribir la salida; el reproceso requiere `--force`. Antes de avanzar, el servidor verifica que todos los rangos anteriores estén **done**.

### 6.2. Cinco jobs Flink

| N.º | Job                    | Salida         | Descripción                                                                                    |
|-----|------------------------|----------------|------------------------------------------------------------------------------------------------|
| F1  | Normalización          | Resultados NLP | Un resultado principal por comentario; confirma offsets en <code>flink-job1-normalize</code> . |
| F2  | Métricas por ventana   | Resultados NLP | Ventanas de tiempo con volumen, autores únicos, longitud media, vacíos y spam aparente.        |
| F3  | Señales políticas      | Resultados NLP | Reglas de terruqueo, fraude, instituciones, política, discriminación e insultos.               |
| F4  | Polarización por actor | Resultados NLP | Agrupar menciones, insultos y señales por actor en ventanas temporales.                        |
| F5  | Alertas de riesgo      | Alertas        | Emite solo casos que superan condiciones de alerta, con severidad y explicación.               |

### 6.3. Cinco jobs Spark

| N.º | Job                    | Entrada/salida         | Descripción                                                                              |
|-----|------------------------|------------------------|------------------------------------------------------------------------------------------|
| S1  | Kafka Raw Ingest       | Kafka → Parquet        | Lee offsets y filtra <code>row_number</code> por rango; exige exactamente 1,000 filas.   |
| S2  | Reglas locales         | Parquet → reglas       | Genera flags multilabel, score local y reporte de distribución.                          |
| S3  | Entrenamiento OffendES | OffendES → modelos     | Entrena y persiste clasificadores binario y multiclase con Spark ML. Se ejecuta offline. |
| S4  | Inferencia OffendES    | Parquet + modelos → ML | Aplica ambos modelos a comentarios Kafka y conserva probabilidades y clases.             |
| S5  | Híbrido y agregados    | Reglas + ML → Curated  | Calcula nivel y razón de riesgo, agrega métricas y produce reportes S3.                  |

## Capítulo 7

# Reporte de métricas

### 7.1. Validación distribuida de 2,000 eventos

| Métrica                                             | Resultado  |
|-----------------------------------------------------|------------|
| Voters Kafka / leader                               | 3 / activo |
| Factor de réplica / minimum ISR                     | 3 / 2      |
| Particiones under-replicated                        | 0          |
| Particiones offline                                 | 0          |
| raw_youtube_chat                                    | 2,000      |
| nlp_stream_results                                  | 4,021      |
| alerts_polarization                                 | 10         |
| Lag total Flink                                     | 0          |
| Nodos YARN compute                                  | 2          |
| Batches Spark completos / fallidos                  | 2 / 0      |
| Solapamiento de <code>event_id</code> entre batches | 0          |

Cuadro 7.1: Evidencia registrada en la validación distribuida.

### 7.2. Métricas ML

| Modelo              | Accuracy | Macro F1 |
|---------------------|----------|----------|
| OffendES binario    | 0.838968 | 0.749030 |
| OffendES multiclase | 0.771939 | 0.548677 |

La inferencia ML cruda marcó 53.25 % como ofensivo en YouTube, pero 60.31 % de las predicciones se concentró en confianza 0.50–0.70. Por cambio de dominio, ese porcentaje no se adopta como conclusión final.

### 7.3. Métricas del corpus completo

Sobre 160,464 comentarios, las reglas registraron 4.05 % de terruqueo, 5.03 % de fraude, 4.46 % de instituciones electorales, 31.36 % de menciones políticas, 5.70 % de polarización y 3.21 % de insulto general. El score local promedio fue 0.5698, con máximo 8.

El scoring híbrido produjo:

| Nivel | Porcentaje |
|-------|------------|
| Bajo  | 52.37 %    |
| Medio | 42.44 %    |
| Alto  | 5.19 %     |

## 7.4. Cadencia del dashboard

| Dato                         | Frecuencia objetivo | Fuente          |
|------------------------------|---------------------|-----------------|
| RAW, eventos y normalizados  | 3 s                 | live delta      |
| Estado Spark                 | 3 s                 | estados remotos |
| Inventario Kafka y threshold | 5 s                 | offsets Kafka   |
| Salud integral               | 5 s                 | snapshot + SSH  |

**Flink normalizados** se obtiene del offset confirmado por `flink-job1-normalize`. De este modo el contador representa progreso real, queda limitado por RAW y no aumenta al recargar o volver a entregar eventos visuales.

## Capítulo 8

# Interfaz gráfica funcional

La interfaz web implementa:

- arranque asíncrono, progreso y parada distribuida;
- chat RAW y eventos Flink en vivo;
- tarjetas de muestra válida, normalizados, alertas y filas Spark;
- búsqueda y filtros de comentarios;
- polarización por actor y alertas;
- inventario de batches y comentarios ofensivos por batch;
- quorum Kafka, topics, lag, YARN, workers y acciones de recuperación.

El estado principal usa tres situaciones:

1. **sin conexión:** no existe sesión activa;
2. **consultando:** se está obteniendo el primer diagnóstico;
3. **conectado:** sesión AWS activa, texto y foco verde.

Una condición técnica degradada no cambia el encabezado a “desconectado”: se conserva en las acciones de diagnóstico. Si Kafka deja de responder, el último estado válido permanece visible y la plataforma nunca se reinicia automáticamente.



## Capítulo 9

# Capturas del dashboard en operación

Este capítulo documenta la interfaz web durante una sesión activa sobre AWS. Las capturas corresponden a una corrida con más de 75 000 comentarios ingeridos desde Kafka y tres batches Spark completados. Cada figura ilustra una capa distinta del plano de observabilidad: visión global del pipeline, streaming Flink con reglas y procesamiento batch con ML.

### 9.1. Vista general y chat en vivo

La primera captura muestra la pantalla principal del *Radar electoral de comentarios en vivo*. En la cabecera se resume la ruta de datos S3 RAW → EMR\_PRIMARY / KAFKA → EMR\_WORKERS / FLINK + SPARK y los controles operativos: pausar la cinta de reproducción, detener la plataforma y el indicador de sesión **conectado**.



Figura 9.1: Vista general del dashboard con chat Kafka en vivo, contadores de topics y tarjetas de KPI.

El panel izquierdo confirma que Kafka está activo e informa los offsets acumulados de `raw_youtube_chat` (75 426), `normalized_comments` (75 426) y `alerts_polarization` (1 818). El panel derecho, titulado *Chat Kafka en vivo*, despliega el feed RAW con autor y texto de cada comentario electoral peruano.

Las cuatro tarjetas inferiores sintetizan el progreso del pipeline:

- **Muestra validada:** total de mensajes publicados en Kafka y visibles para Flink y Spark.
- **Flink normalizados:** un resultado principal por comentario, alineado con el offset confirmado del job 1.
- **Alertas:** eventos emitidos en `alerts_polarization` por el job Flink 5.
- **Spark batch:** filas ya curadas en S3 a partir de rangos disjuntos de 1 000 eventos (3 000 en esta sesión).

La barra inferior permite filtrar mensajes procesados por palabras clave (*ONPE*, *terruqueo*, *Keiko*, etc.), conectando la observabilidad en tiempo real con el análisis cualitativo del corpus.

## 9.2. Eventos Flink y comentarios filtrados por reglas

La segunda captura divide la pantalla en dos flujos complementarios del procesamiento streaming.

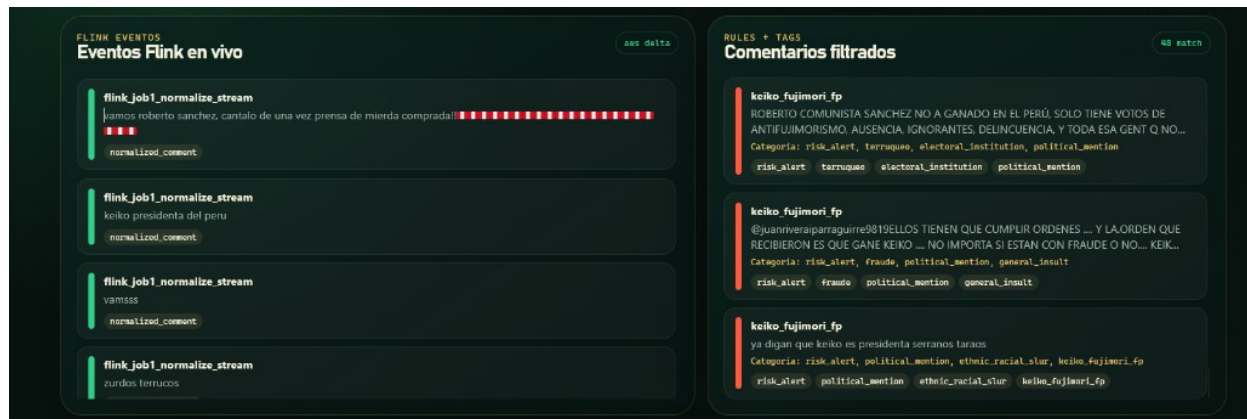


Figura 9.2: Panel de eventos Flink normalizados (izquierda) y comentarios filtrados por reglas y etiquetas (derecha).

En el panel *Eventos Flink en vivo* (etiqueta `aws delta`) aparecen salidas del job `flink_job1_normalize_stream`. Cada tarjeta conserva el texto normalizado y la etiqueta `normalized_comment`, evidenciando la capa de ingestión de baja latencia antes de aplicar lógica política.

El panel *Comentarios filtrados* (RULES + TAGS) muestra coincidencias del motor de reglas locales. Los registros provienen de reglas por actor (`keiko_fujimori_fp`) y exponen categorías detectadas: `risk_alert`, `political_mention`, `terruqueo`, `electoral_institution`, `fraude`, `general_insult` y `ethnic_racial_slur`. El contador `48 match` resume cuántos comentarios del subconjunto visible activaron al menos una regla explicable, sin depender del modelo ML.

Esta vista demuestra la trazabilidad entre normalización streaming y señales interpretables del contexto electoral peruano.

## 9.3. Procesamiento Spark batch y filtrado ML

La tercera captura corresponde al módulo *Spark Batch Enrichment*, donde el dashboard supervisa batches de 1 000 eventos y la inferencia OffendES sobre cada rango completado.

La cabecera resume **RAW visto** (82 929), **DATA SIZE** (100 000), **BATCH SIZE** (1 000) y **Activos / límite** (1/1), coherente con `SPARK_MAX_CONCURRENCY=1`. La cuadrícula central muestra el ciclo de vida de cada batch:

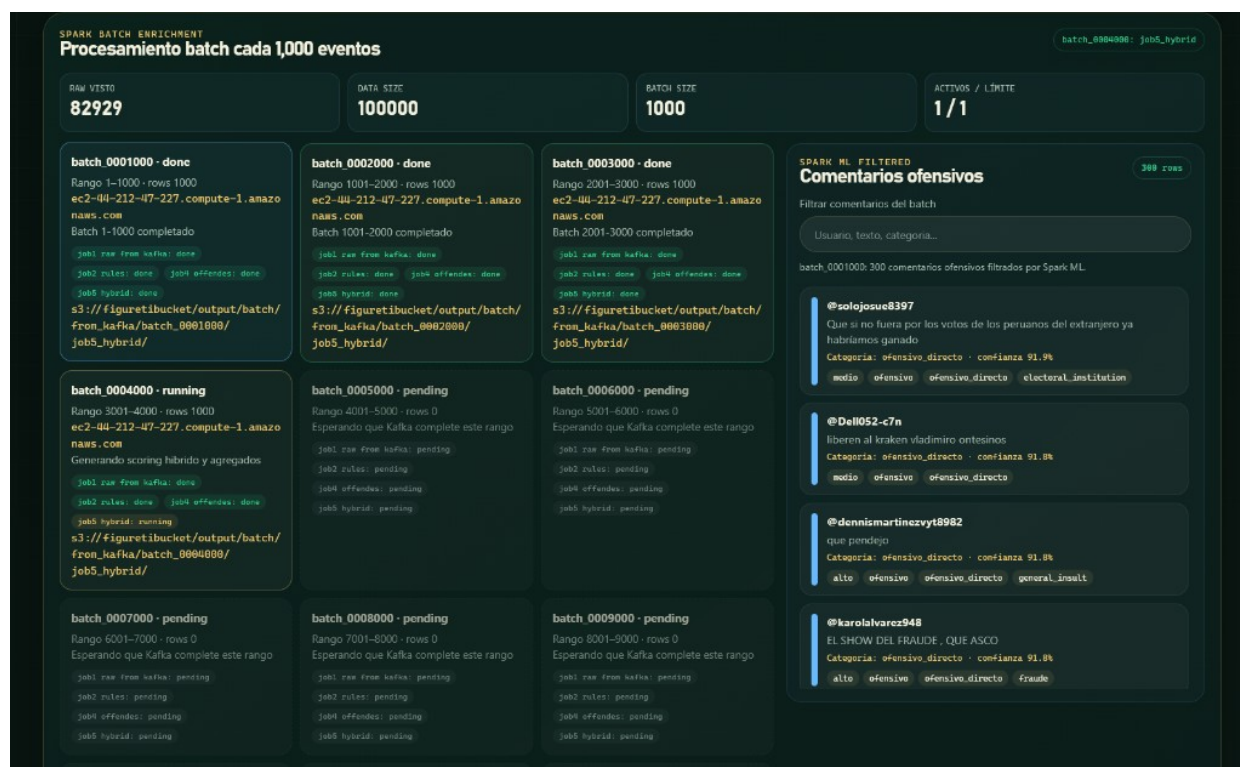


Figura 9.3: Cuadrícula de batches Spark con sub-jobs por rango y panel lateral de comentarios ofensivos detectados por ML.

- **done** (verde): batch\_0001000 a batch\_0003000 finalizaron los cuatro sub-jobs (job1 raw from kafka, job2 rules, job4 offends, job5 hybrid) y escribieron Parquet en rutas S3 exclusivas.
- **running** (amarillo): batch\_0004000 ejecuta el scoring híbrido y agregados.
- **pending** (gris): batches posteriores esperan que Kafka complete el rango de `row_number` correspondiente.

El panel lateral *Spark ML filtered: Comentarios ofensivos* lista predicciones del job 4 para batch\_0001000: usuario, texto, categoría (*ofensivo\_directo*), confianza (p.ej. 91,9 %) y etiquetas de riesgo (*medio*, *alto*, *ofensivo*, *electoral\_institution*, *fraude*). El resumen indica 300 comentarios ofensivos filtrados en ese batch, ilustrando cómo el dashboard cruza progreso operativo batch con evidencia cualitativa del modelo supervisado.

## Capítulo 10

# Conclusiones

1. La separación entre Kafka y compute mejora el aislamiento de recursos y permite escalar el procesamiento sin comprometer el bus.
2. Los grupos Flink estables y los rangos Spark disjuntos proporcionan trazabilidad reproducible desde Kafka hasta S3.
3. El modelo OffendES aporta capacidad supervisada, pero el cambio de dominio impide usar su salida cruda como conclusión política.
4. Las reglas locales hacen explícitas las señales peruanas; el scoring híbrido ofrece una interpretación más defendible.
5. La operación requiere parada confirmada y concurrencia Spark controlada. La idempotencia evita relanzamientos por recargas del navegador.
6. El dashboard ya es funcional como plano de control y observabilidad. El contador normalizado se basa ahora en offsets confirmados, no en mensajes visuales repetidos.

## Limitaciones y trabajo futuro

- Incorporar anotación humana peruana para medir precisión local y sesgo.
- Separar modelos de toxicidad, odio, ofensividad y polarización.
- Persistir series de latencia extremo a extremo para percentiles p50/p95/p99.
- Automatizar creación y terminación de clústeres con infraestructura como código.
- Añadir autenticación al plano de control si deja de ser exclusivamente local.

## Apéndice A

# Trazabilidad documental

La arquitectura vigente se documenta en `README.md`, `architecture.md`, `README_AWS_SETUP.md`, `docs/comandos_levantar_desde_cero.md` y los documentos de `emr_kafka_setup/docs`. Los reportes de pruebas pequeñas se conservan como evidencia histórica y están marcados para no contradecir la política actual.

## Apéndice B

# Rutas principales

```
emr_kafka_setup/dashboard/server.js  
emr_kafka_setup/dashboard/app.js  
emr_kafka_setup/dashboard/scripts/bootstrap_emr_streaming.sh  
emr_kafka_setup/dashboard/scripts/stop_emr_streaming.sh  
emr_kafka_setup/flink/jobs/FlinkKafkaStreamingJobs.java  
emr_kafka_setup/spark/jobs/  
doc/informe_final.tex
```